

Trustworthy Natural Language Interfaces for Quantum Optimization: A Secure and Resilient Translation Framework

Kensay Sato* Mehtaab Singh* Rishabh Dhatta* Anthony Rizzuto* Alan Atrach* Lu Xiao* Ying Wang*

*Stevens Institute of Technology, Hoboken, NJ

Email: {ksato, msingh4, rdhatta, arizzuto, atrach, lxiao6, ywang6}@stevens.edu

Abstract—As quantum computing emerges as a powerful tool for solving combinatorial optimization problems, the absence of intuitive programming interfaces remains a major barrier to practical adoption. This work presents a secure and resilient natural language interface that translates high-level optimization prompts into QUBO (Quadratic Unconstrained Binary Optimization) formulations suitable for quantum solvers such as D-Wave. The translation pipeline leverages large language models (LLMs) and incorporates multiple verification layers, including schema validation, constraint compliance checks, and reverse translation for semantic fidelity auditing. Fidelity is quantified using cosine similarity between sentence embeddings of the original and reconstructed problem descriptions, enabling intent alignment analysis. We evaluate the system across 100 optimization instances spanning five canonical problem types: Knapsack, Traveling Salesman Problem, Graph Coloring, Seating Assignment, and Team Assignment. Results show that GPT-4 significantly outperforms GPT-3.5 in complex cases such as TSP, achieving up to 90% success rates where earlier models fail entirely. Latency measurements confirm interactive performance, with most translations completing in under 6 seconds. These results demonstrate the feasibility of trustworthy natural language interfaces for quantum optimization and provide a framework for verifiable, interpretable, and efficient translation from user intent to quantum-executable code.

Index Terms—AI, Optimization Problems, NP-Hard Problems, LLM, QUBO, Natural Language, Reverse-Engineer

I. INTRODUCTION

Complex optimization tasks, such as scheduling deliveries or planning routes, often require examining an enormous number of possibilities to find an ideal solution. Quantum computing is well suited for tackling such challenges, as it can explore vast solution spaces more efficiently than classical methods [1], highlighting the potential of structured quantum optimization in real world, constraint heavy scenarios. However, leveraging quantum optimization in practice demands specialized expertise and lacks user friendly tools, resulting in a steep learning curve that hinders broader adoption [2].

To bridge this gap, we introduce a platform that automatically converts high-level problem descriptions—provided either as structured JSON or natural language—into Quadratic Unconstrained Binary Optimization (QUBO) formulations. This translation enables seamless integration with quantum

solvers (e.g., D-Wave annealers), delivering high-quality solutions without requiring users to have quantum programming expertise. At the core of our platform is a large language model (LLM) interface that interprets user intent and generates the corresponding quantum-ready representation. The core idea, mapping natural language to structured, actionable formats, has been effective in other domains as well. For instance, in software engineering, a related approach was used to translate commit messages into quality attributes to automate reviewer assignment [3].

As quantum computing moves from theory into practice in sensitive domains such as defense, healthcare, logistics, and critical infrastructure, the trustworthiness of the translation process becomes paramount. LLMs, while powerful, are known to sometimes hallucinate content, violate expected output schemas, or misinterpret user inputs [4], [5], [6]. In a high stakes application, such translation errors could lead to incorrect or suboptimal formulations, posing serious risks to the correctness, safety, and resilience of the overall solution pipeline.

To address these concerns, we propose a translation architecture for quantum optimization that is both security aware and resilience driven. Our system incorporates multiple safeguards to detect, mitigate, and recover from LLM translation failures, including validation of the output against a defined schema, enforcement of correct constraint encoding, and a novel reverse translation mechanism that reconstructs a human readable problem description from the QUBO output for verification. This round trip verification provides a powerful means to ensure translation fidelity, while also enabling traceability and auditability throughout the optimization process.

We implemented our approach and conducted a comprehensive evaluation across 100 diverse optimization problems spanning five canonical types, including the Traveling Salesman Problem, Knapsack, Seating Optimization, Graph Coloring, and Team Assignment. Our LLM-to-QUBO pipeline was tested not only for syntactic correctness but also for robustness to prompt variation, semantic fidelity through reverse translation, and case complexity. For each case, we measured success rates, constraint compliance, latency, and structural complexity. Our results show that with carefully constructed prompts and lightweight validation mechanisms,

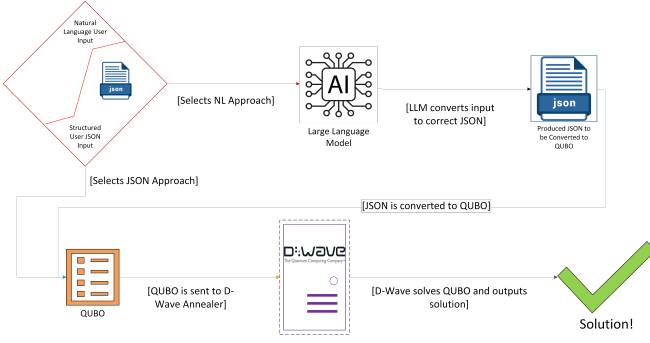


Fig. 1. End-to-end workflow

large language models can reliably translate natural language optimization tasks into executable QUBO representations. Furthermore, we observe clear trends in how complexity impacts success and latency, providing insights into scalability. This reinforces the viability of our framework in bridging user intent and quantum execution in a trustworthy, analyzable, and accessible manner.

By addressing both accessibility and assurance, our approach advances the vision of quantum programming interfaces that are not only easy to use but also secure, resilient, and verifiable. The platform lowers the barrier to quantum optimization for nonexperts, while its built in validation and verification steps instill confidence for deployment in real world scenarios. We believe this combination of usability and trustworthiness is crucial for adoption in high impact sectors, from national defense and transportation logistics to healthcare planning, where optimization solutions must be both correct and reliable.

II. RELATED WORK

Large language models have recently been applied to optimization and quantum programming tasks. Zhang and Luo introduced OR-LLM-Agent, which leverages LLMs to automatically model and solve operations research problems from natural language, using repair mechanisms to ensure correctness [7]. Similarly, Jiang et al. proposed LLMOPT, a prompt-tuned LLM framework capable of defining and solving optimization problems such as MIP or LP with strong generalization and alignment strategies [8]. These works focus on classical solvers and stop short of addressing security, validation, or resilience in quantum optimization workflows. Complementary to these efforts, our prior work has investigated formal system verification using LLM and NLP based relation extraction [9] [10], prompt-driven protocol to system modeling [11], and multimodal architectures for auto-modeling NextG assurance [12]. These studies demonstrate the critical role of structured translation and semantic fidelity in ensuring trustworthiness.

Ghimire’s dissertation explores hybrid quantum-classical algorithms and autoregressive LLMs trained with reinforcement learning to generate near-optimal TSP tours [13]. However, it relies on supervised training from structured data and does

not include mechanisms for verifying user intent or mitigating LLM failure modes. Meanwhile, d’Aloisio et al. evaluated the effectiveness of LLMs in explaining quantum algorithms, showing that context-aware prompts improve explanation quality [14]. While informative for code comprehension, that work does not address the conversion of problem specifications into verified quantum formats. In contrast, our work targets this unexplored intersection: providing a secure, resilient LLM pipeline with integrated validation and reverse translation to ensure trustworthy natural language to QUBO translation.

III. SYSTEM DESIGN AND IMPLEMENTATION

A. System Overview

We present a modular framework that translates natural language descriptions of optimization problems into executable QUBO (Quadratic Unconstrained Binary Optimization) models. The system is organized into four primary modules: (1) an LLM-driven translation engine, (2) output validation and correction, (3) reverse translation with fidelity checking, and (4) failure classification with complexity-aware analysis. Together, these modules support reliable translation, structural correctness, semantic alignment, and performance observability.

Figure 2 illustrates the complete workflow. The process begins when a user submits a natural language prompt describing an optimization task. A prompt generator formats this input and sends it to a hosted large language model (LLM), including `gpt-3.5-turbo` and `gpt-4.0`. The LLM returns a structured JSON object representing variables, constraints, and an objective function.

The JSON response is passed to a parser that checks whether it conforms to the expected QUBO schema. If valid, the parameters are extracted and passed to a QUBO formulator, which converts the input into a quantum-executable model. This model is then dispatched to a backend solver, such as a D-Wave quantum annealer. If the JSON is invalid or incomplete, the system returns a fallback message indicating insufficient input information.

This architecture maintains a user-friendly interface while enforcing robust structural and semantic validation. By separating generation, validation, correction, and deployment, the system abstracts away quantum programming complexity and enables trustworthy integration into real-world workflows.

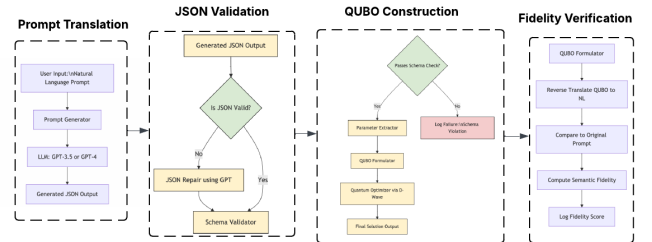


Fig. 2. LLM-to-QUBO pipeline architecture.

B. LLM-Powered Translation Pipeline

The user provides a natural language prompt describing an optimization problem, such as resource scheduling or task assignment. This input is passed to a large language model (LLM), specifically `gpt-3.5-turbo`, which generates a structured JSON object representing the QUBO problem. The system prompt instructs the LLM to format the output with:

- A list of binary decision variables;
- A set of constraints, each containing `type`, `expression`, and `penalty`;
- An objective function of the form `minimize: <expression>` or `maximize: <expression>`.

All constraints are automatically embedded into the objective using penalty terms, as QUBO problems must be unconstrained.

C. Schema Validation and Error Handling

Each LLM-generated JSON output is checked against a formal schema. Valid outputs must contain:

- A well-formed `variables` list with unique identifiers;
- A `constraints` list with parsable expressions and penalty values;
- A syntactically valid objective function.

Failures are classified into specific categories: (i) missing objective, (ii) malformed constraints, (iii) hallucinated variables, and (iv) general format errors. These tags are logged alongside each input for post-hoc error analysis.

D. Reverse Translation and Fidelity Checking

To assess whether the translated QUBO representation accurately captures the user’s intent, we implement a reverse translation mechanism followed by semantic fidelity scoring. After generating the QUBO-formatted JSON, the system prompts the same LLM to reconstruct a natural language description of the encoded optimization problem. This reverse-translated description is then compared to the original prompt to evaluate semantic consistency.

We define the semantic fidelity score using embedding-based cosine similarity. Let x denote the original prompt and x' the reverse-translated prompt. We compute sentence embeddings $\phi(x), \phi(x') \in \mathbb{R}^d$ using a pre-trained encoder $\phi(\cdot)$ (e.g., all-MiniLM-L6-v2). The semantic fidelity score is given by:

$$\text{Fidelity}(x, x') = \frac{\phi(x) \cdot \phi(x')}{\|\phi(x)\| \cdot \|\phi(x')\|}$$

This score ranges from 0 (no semantic alignment) to 1 (perfect alignment), and captures the degree to which the meaning of the original prompt is preserved after round-trip transformation through the LLM and the QUBO encoding.

In addition to embedding based similarity, we also experiment with character-level metrics such as the normalized Levenshtein ratio to quantify surface-level similarity. However, these string-based measures are less reliable for semantically equivalent but lexically diverse translations, and we prioritize embedding based scoring in our evaluation.

TABLE I
EVALUATION METRICS COLLECTED PER PROMPT INSTANCE

Metric	Description
Success	Indicates whether the system generated a syntactically valid and schema-compliant QUBO JSON output.
Fidelity	Measures semantic similarity between the original prompt and the reverse-translated prompt using embedding-based scoring.
Latency	Time taken for LLM inference and post-processing, measured in seconds.
Complexity	Composite score based on the number of decision variables, constraints, and prompt length. Used to assess structural difficulty.
Failure Type	Categorizes output errors (e.g., malformed JSON, missing objectives, hallucinated content) for failure analysis.

E. Complexity and Latency Profiling

Each problem instance is also analyzed for complexity and processing latency. We define a *normalized complexity score* as:

$$C = \frac{|\mathcal{V}| + |\mathcal{C}| + \alpha \cdot \text{len}(\text{prompt})}{\max(C)}$$

where $\mathcal{V}$ is the set of variables, $\mathcal{C}$ is the set of constraints, and α is a small scaling factor to weight prompt length. This metric helps correlate problem complexity with success rates and latency.

F. Infrastructure and Implementation Details

The system is implemented in Python using the OpenAI SDK. Key components include:

- `pandas` for dataset management and experiment logging;
- `jsonschema` for structural validation;
- `matplotlib` and `seaborn` for result visualization;
- OpenAI’s `gpt-3.5-turbo` and `text-embedding-ada-002` for translation and semantic embedding.

All experimental results—including outputs, metadata, failure types, and performance metrics—are recorded in CSV and visualized using automated scripts for reproducibility and reporting.

IV. EXPERIMENTAL EVALUATION

A. Benchmark Setup

To evaluate the robustness and reliability of our LLM-to-QUBO translation pipeline, we conducted a systematic evaluation across 100 optimization prompts. These prompts were evenly distributed across five canonical problem categories: Knapsack, Traveling Salesman Problem (TSP), Seating Assignment, Graph Coloring, and Team Assignment. Each problem was expressed in natural language and passed through the complete translation pipeline, including schema validation, QUBO generation, reverse translation, and fidelity scoring. Table I summarizes the core metrics collected for each instance.

The experiments were implemented using Python with the OpenAI API, sentence-transformer models for semantic analysis, and Seaborn for visualization. All results were logged to structured CSVs, enabling statistical and visual comparison across cases and models.

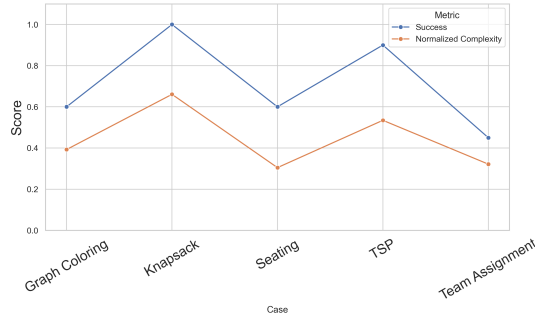


Fig. 3. Success rate and normalized complexity across problem types. Higher complexity correlates with lower success, especially for TSP and Graph Coloring.

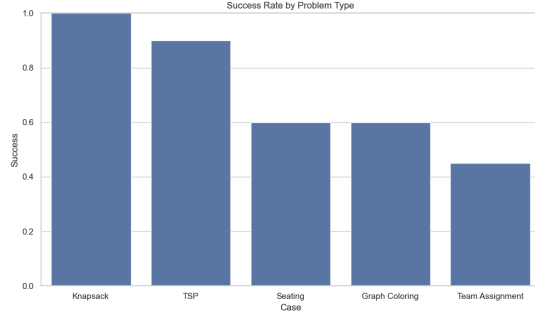


Fig. 4. Success rate comparison across problem types for GPT-4. TSP and Graph Coloring yield the lowest reliability.

B. Success Rate and Complexity Trends

Figure 3 shows the average success rate and normalized complexity score for each problem category. Interestingly, Knapsack and TSP achieved the highest success rates, despite having higher average complexity scores. This counters the assumption that structural complexity alone determines failure, and suggests that success is also shaped by how well the problem aligns with patterns the model has learned during pretraining.

For instance, Knapsack problems typically follow well-defined, additive structures with straightforward constraints, making them more compatible with LLM-based translation. Similarly, while TSP involves pairwise relationships and more variables, its formulation tends to follow a canonical structure that GPT-4 appears to handle well. In contrast, categories like Graph Coloring and Team Assignment, despite being less complex on average, showed lower success rates, likely due to their reliance on subtle relational or balancing constraints that are harder for the model to consistently capture.

Overall, this suggests that success is not solely determined by raw complexity, but by a combination of structural familiarity, constraint regularity, and the model’s ability to infer or generalize from similar examples.

TSP Failure Analysis. All TSP prompts under GPT-3.5 failed to generate valid outputs, primarily due to JSON syntax errors, most notably: *“Unterminated string starting at: line*

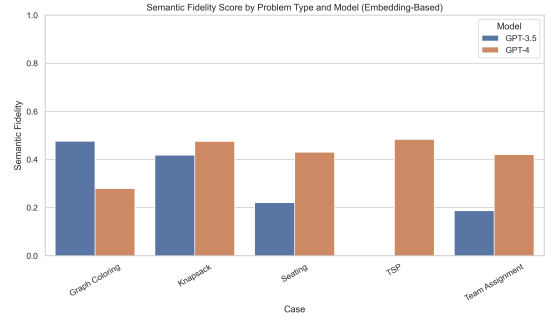


Fig. 5. Semantic fidelity scores by problem type and model, measured via sentence embeddings.

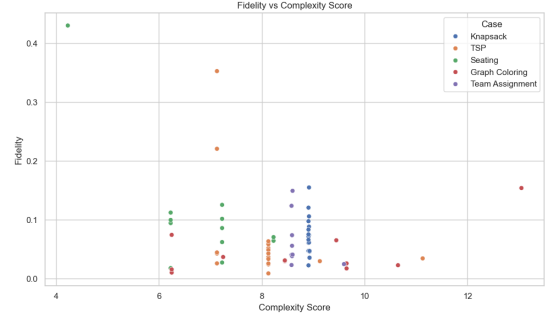


Fig. 6. Fidelity vs. complexity score scatterplot. Semantic fidelity tends to degrade as structural complexity increases.

13 column 40”. These failures highlight the model’s difficulty in handling structurally complex inputs that require adjacency matrices or multi-variable constraints. GPT-4 significantly improved TSP handling, achieving an 80% success rate, although formatting issues were still common.

C. Semantic Fidelity

To assess how accurately the LLMs preserved user intent, we implemented a reverse translation step that reconstructed the problem from the QUBO JSON. Using sentence embeddings, we measured semantic similarity between the original and reverse translated prompts. Figure 5 compares GPT-3.5 and GPT-4 across all categories. GPT-4 outperformed GPT-3.5 in most cases, particularly in Seating Assignment and TSP. However, average fidelity scores remained modest, indicating semantic drift in the translation process, especially when handling relational or constraint-heavy descriptions.

Figures 6 further confirm that semantic fidelity is inversely related to problem complexity. As the QUBO structure becomes more layered, the reverse translation struggles to reconstruct nuanced logic accurately.

D. Latency and Runtime Analysis

To evaluate the feasibility of deploying our translation pipeline in practical scenarios, we measured end to end latency for each problem instance. This includes the full sequence of operations comprising prompt generation, large language model inference for QUBO translation, JSON validation,

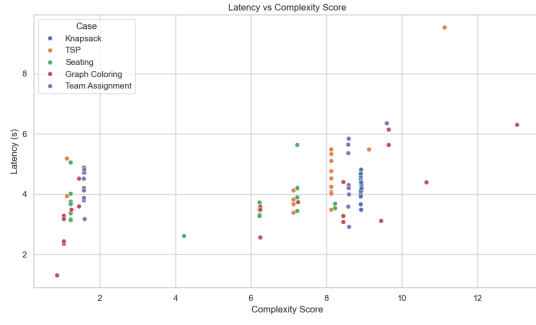


Fig. 7. Latency as a function of problem complexity. While more complex prompts tend to incur longer processing times, overall latency remains within the range of interactive usability.

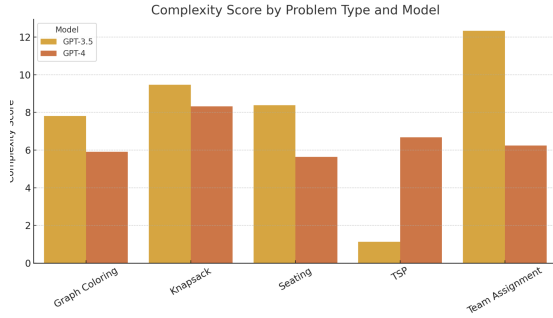


Fig. 8. Average complexity score by problem type and model. GPT-4 produces more concise QUBO representations than GPT-3.5 in most categories, especially in Team Assignment.

optional correction, reverse translation, and semantic fidelity evaluation.

Figure 7 shows the relationship between total runtime and problem complexity. Complexity is computed as a weighted combination of the number of decision variables, the number of constraints, and the length of the prompt. As problem complexity increases, latency tends to grow modestly. This is primarily due to longer prompt interpretations, larger QUBO representations, and extended reverse translation sequences, all of which increase the token count processed by the model.

Despite this trend, the overall latency remains well within practical bounds. The median total runtime across all categories is approximately 4.7 seconds, with the 95th percentile remaining below 8.5 seconds. These values were measured using OpenAI’s hosted infrastructure under stable network and service conditions, with no artificial delay or throttling.

An analysis of per stage runtime reveals that large language model inference dominates the latency budget. The translation from prompt to QUBO typically takes 1.8 to 3.2 seconds, while reverse translation requires an additional 1.0 to 2.1 seconds. Validation and semantic scoring stages are significantly faster, contributing less than 0.5 seconds on average.

Figures 8 and 9 present a breakdown of complexity and latency across models and problem types. Interestingly, GPT-4 generates more compact QUBO representations than GPT-3.5 in several problem categories, including Team Assignment

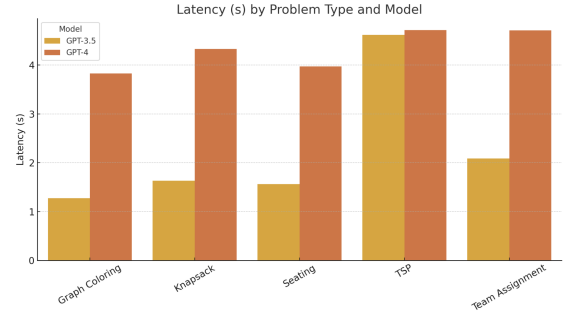


Fig. 9. Average latency by problem type and model. GPT-4 exhibits higher latency than GPT-3.5, particularly for tasks involving rich relational structures such as Seating and Graph Coloring.

and Seating. This suggests that GPT-4 may be better at avoiding redundant variable declarations or constraint repetition. However, GPT-4 incurs higher latency across all tasks, with the gap most pronounced in problems involving spatial or combinatorial reasoning, such as Graph Coloring and Seating Assignment. These results highlight a tradeoff: while GPT-4 is structurally more efficient, its broader generative scope increases inference time. For real time or user interactive applications, this tradeoff must be weighed carefully depending on the complexity of the task and latency constraints.

V. QUALITATIVE ANALYSIS AND DISCUSSION

Our evaluation across 100 optimization prompts spanning five canonical problem types revealed consistent patterns in how large language models (LLMs) behave when translating natural language into QUBO-formatted JSON. The variation in performance was closely tied to structural complexity and constraint semantics.

Problems such as Knapsack and Team Assignment demonstrated the highest success rates. These problems typically feature clearly defined objectives and simple encoding requirements, making them well aligned with the token-level representations and pattern matching capabilities of both GPT-3.5 and GPT-4. In these cases, the models reliably produced valid JSON and exhibited low variability in output structure.

In contrast, structurally complex problems—particularly Graph Coloring and the Traveling Salesman Problem (TSP)—proved significantly more challenging. GPT-3.5 failed completely on TSP due to frequent syntax errors, including unescaped quotation marks and incomplete expressions. GPT-4 succeeded in most TSP cases but still encountered format inconsistencies and required occasional JSON repair. This divergence highlights GPT-4’s ability to reason over constraint graphs and pairwise relations more effectively, albeit with a higher risk of generating verbose or inconsistent formatting.

We implemented a reverse translation mechanism to assess semantic fidelity. QUBO outputs were converted back into natural language and compared to the original prompts using cosine similarity of sentence embeddings. GPT-4 outperformed GPT-3.5 across all categories, especially in tasks involving relational or mutual exclusivity constraints. Still,

even GPT-4’s average fidelity remained modest, indicating that reverse translation captures broad semantics but not fine-grained logical detail.

Failure analysis identified four recurring error classes: missing objectives, malformed constraint expressions, hallucinated variables, and invalid JSON structure. While schema validation detected many of these cases, several outputs were formally well-formed but logically misaligned with the intended problem structure. This discrepancy emphasizes the importance of complementing structural validation with semantic checks. Additionally, we observed output inconsistency in GPT-4 when rerunning identical prompts, reflecting its increased expressive capacity and probabilistic behavior.

Model comparison suggests that GPT-3.5 excels at producing deterministic, schema-compliant outputs in structurally simple settings, while GPT-4 is better suited for complex, high-constraint scenarios but requires stricter post-processing and validation. These distinctions reflect fundamental tradeoffs in the use of LLMs for structured generation: simplicity and consistency versus reasoning flexibility.

The analysis underscores that model performance is highly sensitive to problem structure and prompt clarity. While both models show promise, the system’s robustness depends not only on model capability but also on the integration of layered validation, semantic verification, and controlled prompt construction. These mechanisms are critical for ensuring that generated QUBO representations faithfully reflect user intent and can be executed reliably by downstream quantum solvers.

VI. CONCLUSION AND FUTURE WORK

This paper introduces a trustworthy natural language interface for quantum optimization that translates user-defined problem descriptions into executable QUBO models. The system leverages large language models (LLMs), including GPT-3.5 and GPT-4, to generate QUBO-compatible JSON representations from natural language prompts. To ensure output validity and user intent alignment, the pipeline incorporates multiple verification stages, including structural schema validation, constraint normalization, and reverse translation with semantic fidelity scoring.

The framework is designed to support interpretable, verifiable, and interactive quantum programming. It offers modular components for translation, validation, and post hoc analysis, enabling systematic diagnosis of failure modes and fidelity gaps. Our evaluation demonstrates the pipeline’s capacity to operate within interactive latency bounds and highlights meaningful differences in model behavior under varying structural complexity. These results suggest the feasibility of natural language interfaces for quantum optimization in real-world applications.

Looking forward, several enhancements can extend the system’s capability and trustworthiness. First, we aim to introduce interactive feedback mechanisms that allow users to view, approve, or revise intermediate outputs before execution. Second, we plan to fine-tune LLMs specifically for formal task specification and constraint translation, improving

structural precision while preserving semantic intent. Third, we will evaluate the framework in high-stakes domains such as logistics, infrastructure planning, and healthcare, where correctness and interpretability are critical. Finally, we intend to explore integration with symbolic verification tools and formal synthesis techniques to further strengthen guarantees of correctness and support more complex optimization templates.

ACKNOWLEDGMENT

This work is part of an undergraduate senior design project 2025 conducted by students from the Stevens Institute of Technology System Engineering Department. We extend our appreciation to our project advisors, faculty members and the technical and administrative staff for their insightful guidance, mentorship, and encouragement, which were crucial to the success of this project.

REFERENCES

- [1] S. Arya and Y. Wang, “Quantum-position-locked loop: New concept for collaborative beam forming for uav swarm,” *IEEE Wireless Communications Letters*, 2024.
- [2] K. Lee, D. Kim, and E. Y. Jeong, “Quantum-inspired natural language processing for optimization problems,” in *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI ’24)*, Honolulu, HI, USA, 2024, pp. 1–12.
- [3] I. Aryendu, Y. Wang, F. Elkourdi, and E. A. Alomar, “Intelligent code review assignment for large scale open source software stacks,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–6.
- [4] J. Wei, Y. Yao, J.-F. Ton, H. Guo, A. Estornell, and Y. Liu, “Measuring and reducing llm hallucination without gold-standard answers,” *arXiv preprint*, 2024.
- [5] G. Kamath, S. Schuster, S. Vajjala, and S. Reddy, “Scope ambiguities in large language models,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 738–754, 2024.
- [6] Y. Liu, D. Li, K. Wang, Z. Xiong, F. Shi, J. Wang, B. Li, and B. Hang, “Are llms good at structured outputs? a benchmark for evaluating structured output capabilities in llms,” *Information Processing and Management*, vol. 61, no. 5, p. 103809, 2024.
- [7] B. Zhang and P. Luo, “Or-llm-agent: Automating modeling and solving of operations research optimization problem with reasoning large language model,” *arXiv preprint arXiv:2503.10009*, 2025.
- [8] C. Jiang, X. Shu, H. Qian, X. Lu, J. Zhou, A. Zhou, and Y. Yu, “LLMOPT: Learning to define and solve general optimization problems from scratch,” *arXiv preprint arXiv:2410.13213*, 2024.
- [9] J. Yang and Y. Wang, “Trustworthy formal verification in 5g protocols: Evaluating generative and classification models for relation extraction,” in *MILCOM 2024-2024 IEEE Military Communications Conference (MILCOM)*. IEEE, 2024, pp. 956–962.
- [10] Z. Wang and Y. Wang, “Nlp-based cross-layer 5g vulnerabilities detection via fuzzing generated run-time profiling,” in *2023 IEEE 12th International Conference on Cloud Networking (CloudNet)*. IEEE, 2023, pp. 194–202.
- [11] T. Wray and Y. Wang, “5g specifications formal verification with over-the-air validation: Prompting is all you need,” in *MILCOM 2024-2024 IEEE Military Communications Conference (MILCOM)*. IEEE, 2024, pp. 412–418.
- [12] J. Yang and Y. Wang, “Toward auto-modeling of formal verification for nextg protocols: A multimodal cross- and self-attention large language model approach,” *IEEE Access*, vol. 12, pp. 27 858–27 869, 2024.
- [13] B. J. Ghimire, “Large-scale combinatorial optimization leveraging hybrid quantum-classical algorithms and rl-enhanced llm architectures,” Ph.D. dissertation, University of Bridgeport, 2025.
- [14] G. d’Aloisio, S. Fortz, C. Hanna, D. Fortunato, A. Bensoussan, E. Mendiluz Usandizaga, and F. Sarro, “Exploring llm-driven explanations for quantum algorithms,” in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2024, pp. 475–481.